

assignment_q4_rag_solution_template

May 8, 2026

1 Assignment 2 - RAG (and a bit of Reasoning)

Parts which require your interaction are marked with TODO:

```
[ ]: # make sure we have the relevant dependencies
!pip3 install datasets sentence_transformers tqdm numpy
```

Imagine your task is to build a question-answering (QA) system for a company. You are given a language model and have to create this product out of it. The requirements of the system need to adapt very quickly to the new data without training. For this, we will use Retrieval Augmented Generation (RAG). The company insists you use their in-house LM model trained on multiple tasks, a *flan-t5-small*. You can test its QA functionality by asking the question “When ETH was founded?”:

```
[ ]: # There seems to be an issue with pipelines so we have this custom function.
from transformers import T5ForConditionalGeneration, T5Tokenizer

model = T5ForConditionalGeneration.from_pretrained("google/flan-t5-small")
tokenizer = T5Tokenizer.from_pretrained("google/flan-t5-small")
def vanilla_qa_pipe(text: str) -> str:
    inputs=tokenizer(text, return_tensors="pt")
    outputs = model.generate(**inputs, max_new_tokens=20)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

QUESTION = "When was ETH founded?"
vanilla_qa_pipe(f"QUESTION:{QUESTION} ANSWER:")
```

```
[ ]: vanilla_qa_pipe(f"""
    CONTEXT: ETH Zurich (German: Eidgenoessische Technische Hochschule
    ↪Zurich; English:
    Federal Institute of Technology Zurich) is a public research university
    ↪in Zurich,
    Switzerland. Founded in 1854 with the stated mission to educate
    ↪engineers and scientists,
    the university focuses primarily on science, technology, engineering,
    ↪and mathematics. It
    consistently ranks among the top universities in the world and its 16
    ↪departments span a
```

```

        variety of disciplines and subjects.
        {QUESTION}
        ANSWER:",
        """ ,
    )

```

The first output is 1897, which is incorrect.

In the second example, the model was given context from which it could answer the question. So it gave the right answer. However, if you think of it, it was quite lucky that the context contained the answer it was looking for.

If you are not convinced, look at what happens when we change the question:

```

[ ]: QUESTION = "When was EPFL founded?"
vanilla_qa_pipe(f"""
    CONTEXT: ETH Zurich (German: Eidgenoessische Technische Hochschule
    ↪Zurich; English:
    ↪Federal Institute of Technology Zurich) is a public research university
    ↪in Zurich,
    ↪Switzerland. Founded in 1854 with the stated mission to educate
    ↪engineers and scientists,
    ↪the university focuses primarily on science, technology, engineering,
    ↪and mathematics. It
    ↪consistently ranks among the top universities in the world and its 16
    ↪departments span a
    ↪variety of disciplines and subjects.
    {QUESTION}
    ANSWER:",
    """ ,
)

```

It would be nice if both federal institutes were founded in the same year, but they were not. EPFL was actually established in 1969. Now the obvious question would be, “Why not give the first paragraph of **EPFL**’s wikipedia page as context?

Well, yes, that would actually work. But for the company’s QA system, you cannot sit there looking at every query, searching for the relevant info and feed in.

You do know that customers will only ever ask about things the company works on, and all relevant info is in the company database.

Q1. So why don’t we just feed the model the entire database? (0.5 point)

A1. *TODO*

Okay, so we need something better. We earlier talked about how *you* cannot search for the context. But that does not mean *searching* was a bad idea. You could write code to automatically search the database and find the context. But what would you search for? Well, the only thing we have is the question, so might as well go with that.

```
[ ]: # You only need to run this if you want to use nltk
import nltk
nltk.download('punkt_tab')
from nltk import word_tokenize
# You might need this
word_tokenize("Tokenize this. See what happens.")

[ ]: class basic_search_engine:
    def __init__(self, dataset) -> None:
        self.dataset=list(dataset)
    def get_score(self, question_counts: dict, target: str) -> int:
        # count how many words are shared between the question and the target
        # if a word appears k times in the question and n times in the target
        # it should contribute k*n to the score.
        # Q2.1 TODO (1 points)

    def search(self, question: str, count: int) -> list[str]:
        # return the top <count> contexts in terms of score
        # Q2.2 TODO (1 points)

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than_
↪cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=basic_search_engine(data)
search_engine.search(question, 2)
```

If it worked with flying dogs, it should work with the real data. Test it out.

```
[ ]: import tqdm
import numpy as np
from datasets import load_dataset
dataset = load_dataset("rajpurkar/squad")
context_set=set(dataset["validation"]["context"])
print("Total number of contexts is", len(context_set))
search_engine=basic_search_engine(context_set)

def metric_exact_match(ans_pred: str, ans_true: str) -> float:
    return (ans_pred == ans_true)*1.0

def metric_f1(ans_pred: str, ans_true: str) -> float:
    import collections
    ans_pred = ans_pred.lower().split()
    ans_true = ans_true.lower().split()
    common = collections.Counter(ans_pred) & collections.Counter(ans_true)
    num_same = sum(common.values())
```

```

    if num_same == 0:
        return num_same

    prec = num_same/len(ans_pred)
    recl = num_same/len(ans_true)
    return 2*prec*recl/(prec+recl)

exact_match_scores={}
f1_scores={}
exact_match_scores["search_1"]=[]
f1_scores["search_1"]=[]
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]
    contexts=search_engine.search(question, 1)
    model_output=vanilla_qa_pipe(f"CONTEXT: {contexts[0]}\nQUESTION:
↪{question}\nANSWER:")
    exact_match_scores["search_1"].append(max([metric_exact_match(model_output,
↪a) for a in line["answers"]["text"]]))
    f1_scores["search_1"].append(max([metric_f1(model_output, a) for a in
↪line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.average(exact_match_scores["search_1"]))
print("Average F1 Score:", np.average(f1_scores["search_1"]))

```

Well, Looks like that was neither fast, nor useful. The fact is, searching like this is not quite optimal.

Q3. So what are some issues with the search? Explain with insights from the ETH example (1 point)

A3. *TODO*

Searching a database like this is a non-trivial task and a whole research field of Information Retrieval is devoted to it.

For now, we will pick two off-the-shelf methods, TF-IDF Vectorization and Embedding with Bert.

You might want to look up what these do. Then answer all the following in 1-3 sentences (0.5 point each)

Q4.1. What is TF-IDF?

A4.1. *TODO*

Q4.2. What is the benefit of vectorisation?

A4.2. *TODO*

Q4.3. What is a sentence embedding?

A4.3 *TODO*

Now complete the code below. We have already filled in the initialization

```
[ ]: from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(max_features=768, norm=None)

class tfidf_search_engine:
    def __init__(self, dataset) -> None:
        self.vectorizer = TfidfVectorizer(max_features=768, norm=None)
        self.dataset=list(dataset)
        self.kb_vectorized = self.vectorizer.fit_transform([x for x in self.
↳dataset])

    def search(self, question: str) -> list[str]:
        # return the top contexts in terms of tf-idf score
        # we use cosine similarity, so score(a,b)=vector(a).vector(b)
        # you can use the "transform" function of self.vectorizer
        # q5. TODO (1 point)

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than_
↳cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=tfidf_search_engine(data)
search_engine.search(question)
```

```
[ ]: exact_match_scores["search_tfidf"]=[]
f1_scores["search_tfidf"]=[]
tfidf_engine=tfidf_search_engine(context_set)
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]
    contexts=tfidf_engine.search(question)
    model_output=vanilla_qa_pipe(f"CONTEXT: {contexts}\nQUESTION:
↳{question}\nANSWER:")
    exact_match_scores["search_tfidf"].
↳append(max([metric_exact_match(model_output, a) for a in_
↳line["answers"]["text"]]))
    f1_scores["search_tfidf"].append(max([metric_f1(model_output, a) for a in_
↳line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.
↳average(exact_match_scores["search_tfidf"]))
print("Average F1 Score:", np.average(f1_scores["search_tfidf"]))
```

```
[ ]: from sentence_transformers import SentenceTransformer

class sbert_search_engine:
    def __init__(self, dataset) -> None:
        self.vectorizer = SentenceTransformer("bert-base-nli-mean-tokens").cuda()
        self.dataset=list(dataset)
```

```

        self.kb_vectorized = self.vectorizer.encode(self.dataset)
    def search(self, question: str) -> list[str]:
        # return the top contexts in terms of bert-similarity
        # we use cosine similarity, so score(a,b)=vector(a).vector(b)
        # you can use the "encode" function of self.vectorizer
        # q6. TODO (1 point)

# Testing
data=["All cats can fly", "Some cats can fly planes", "Dogs fly higher than_
↪cats", "Dogs can not fly planes"]
question="Can dogs fly?"
search_engine=sbert_search_engine(data)
search_engine.search(question)

```

```

[ ]: exact_match_scores["search_sbert"]=[]
sbert_engine=sbert_search_engine(context_set)
f1_scores["search_sbert"]=[]
for line in tqdm.tqdm(dataset["validation"].select(range(100))):
    question=line["question"]
    contexts=sbert_engine.search(question)
    model_output=vanilla_qa_pipe(f"CONTEXT: {contexts}\nQUESTION:
↪{question}\nANSWER:")
    exact_match_scores["search_sbert"].
↪append(max([metric_exact_match(model_output, a) for a in_
↪line["answers"]["text"]]))
    f1_scores["search_sbert"].append(max([metric_f1(model_output, a) for a in_
↪line["answers"]["text"]]))
print()
print("Average Exact Match Score:", np.
↪average(exact_match_scores["search_sbert"]))
print("Average F1 Score:", np.average(f1_scores["search_sbert"]))

```

You would note that the scores are still not that high. Perhaps because the similarity metrics are not perfect.

Q7. Calculate how often the top scoring context is in fact the correct context for a) TF-IDF and b) sBert Score (you would have to write some code for this. Only work with the first 100 validation questions) (1.5 point)

A7. TODO

```

[ ]: # write your code for Q7 here
# The 'correct' context for the question dataset["validation"]["question"][i] is
# dataset["validation"]["context"][i]
matches_tf_idf=0
matches_sbert=0
for i in tqdm.tqdm(range(100)):

```

```

if dataset["validation"]["context"][i]==tf_idf_engine.
↪search(dataset["validation"]["question"][i]):
    matches_tf_idf+=1
if dataset["validation"]["context"][i]==sbert_engine.
↪search(dataset["validation"]["question"][i]):
    matches_sbert+=1
print("TF-IDF:", matches_tf_idf/100)
print("SBERT:", matches_sbert/100)

```

Q8. What can we do to account for imperfect similarity metrics? (Hint: if you have been following the code, there was already a hint somewhere) (0.5 point)

A8. *TODO*

You may have noticed that sometimes model can give the right answer despite having the wrong context. But the reverse can also be true.

Q9. For each of the following examples, explain why the model is getting the answer wrong (0.5 point each)

```

[ ]: context="The Berlin Wall (German: Berliner Mauer) was a guarded concrete
↪barrier that encircled West Berlin from 1961 to 1989, separating it from
↪East Berlin and the German Democratic Republic (GDR; East Germany).
↪Construction of the Berlin Wall was commenced by the government of the GDR
↪on 13 August 1961. It included guard towers placed along large concrete
↪walls,[4] accompanied by a wide area (later known as the \"death strip\")
↪that contained anti-vehicle trenches, beds of nails and other defenses. The
↪primary intention for the Wall's construction was to prevent East German
↪citizens from fleeing to the West"
question="For many years did the Berlin wall stand?"
vanilla_qa_pipe(f"CONTEXT: {context}\nQUESTION:{question}\nANSWER:")

```

A9.1: *TODO*

```

[ ]: context="Joseph Robinette Biden Jr.(born November 20, 1942) is an American
↪politician who was the 46th president of the United States from 2021 to 2025.
↪ A member of the Democratic Party, he represented Delaware in the United
↪States Senate from 1973 to 2009 and also served as the 47th vice president
↪under President Barack Obama from 2009 to 2017."
question="Who was the president of United States during 2021-2025?"
vanilla_qa_pipe(f"CONTEXT: {context}\nQUESTION:{question}\nANSWER:")

```

A9.2 *TODO*

#END OF ASSIGNMENT